Banking Management System

Aradhya Chakrabarti (MT2026012)

CS-513 System Software Mini Project
International Institute of Information Technology, Bangalore

1 Functional design

1.1 Purpose and scope

We need to design a banking management system which works over a command line interface. A separate server process should store its data on the filesystem and serve multiple clients over localhost. The client should provide a menu-based interface to the user. The server should be able to process requests from different kinds of users and show the relevant menu items for each role.

1.2 Actors

Actor	Responsibility
Customer	Manages a personal account, transfers funds, applies for a loan, changes the password, views history, and gives feedback.
Employee	Opens and maintains customer accounts, views passbooks, closes eligible accounts, and decides assigned loans.
Manager	Activates or deactivates customers, assigns pending loans, and reviews feedback.
Administrator	Manages staff, user roles, user details, and the audit trail.
System	Handles sessions, permissions, locking, atomic commits, durability, and audit logging.

1.3 Functional requirements

S.No.	Requirement
Authentication and sessions	
AUTH-01	A user logs in with a username and password.
AUTH-02	Passwords are stored as salted hashes, never as clear text.
AUTH-03	A user has at most one active session.
AUTH-04	A deactivated or closed account cannot log in.
AUTH-05	A user can change the password after proving the old one.
AUTH-06	Logout ends the session and keeps the client open.
AUTH-07	Exit logs out first and closes the client.
AUTH-08	A role change or account deactivation revokes the live session.
Customer operations	
CUST-01	The customer can view the current balance.
CUST-02	A deposit increases the balance and writes one ledger row.
CUST-03	A withdrawal decreases the balance and cannot overdraw it.
CUST-04	A transfer debits the sender and credits the recipient in one atomic change.
CUST-05	A customer can apply for a loan with an amount, a term, and a purpose.
CUST-06	The customer can view a passbook of their own transactions.
CUST-07	A transfer appears once for the sender and once for the recipient.
CUST-08	The customer can submit feedback.

S.No.	Requirement
CUST-09	The customer can read manager replies to their feedback.
CUST-10	The customer can view a mini statement.
CUST-11	Customer operations are refused for other roles.
Employee operations	
EMP-01	An employee can add a customer with a unique username and an opening balance.
EMP-02	A duplicate username is rejected.
EMP-03	An employee can change a customer's name or reset the customer password.
EMP-04	The employee can view a customer's passbook.
EMP-05	The employee can list only the loans assigned to them.
EMP-06	The employee can approve or reject an assigned pending loan.
EMP-07	An employee cannot decide a loan assigned to another employee.
EMP-08	An employee can close a customer account with a zero balance and no outstanding loan.
EMP-09	The employee can change their own password.
Manager operations	
MGR-01	A manager can activate or deactivate a customer account.
MGR-02	A manager can assign a pending loan to an active employee.
MGR-03	A manager can list pending loans.
MGR-04	A manager can list feedback and mark it reviewed with a reply.
MGR-05	The manager can change their own password.
Administrator operations	
ADM-01	An administrator can add an employee or a manager.
ADM-02	An administrator can modify any user's name or password.
ADM-03	An administrator can change a user's role among customer, employee, manager, and administrator.
ADM-04	An administrator cannot change their own role.
ADM-05	An administrator can list all users.
ADM-06	An administrator can read the audit trail.
ADM-07	The administrator can change their own password.
ADM-08	An administrator can set the single bank-wide loan interest rate and method. Existing loans keep the policy copied when they were created.
System requirements	
SYS-01	The server serves multiple clients concurrently.
SYS-02	Conflicting reads and writes are serialized.
SYS-03	A multi-record operation commits atomically.
SYS-04	A committed transaction survives a crash.
SYS-05	An uncommitted transaction is not applied.
SYS-06	State-changing operations are audited.
SYS-07	Shutdown stops children, closes IPC objects, and releases the data lock.
SYS-08	A handler does not outlive a crashed server parent.
SYS-09	Only one server process should use a data directory at a time.

1.4 Business rules

Money.

- The system should store every amount as an integer number of paise, where one rupee is 100 paise. This keeps INR amounts exact to two decimal places.
- The system should accept only positive deposit and withdrawal amounts. A balance should never become negative.
- A transfer should debit and credit the same amount. It should reject a transfer to the same account and a recipient who is not an active customer.
- When an employee approves a loan, the system should credit the principal to the customer's balance once. A rejected loan should leave the balance unchanged.

- An account should close only when the balance is zero and there is no pending or approved loan.

Accounts and roles.

- Usernames should be unique and case-sensitive. Each user should have one role.
- An administrator should not change their own role.
- Only active accounts should be able to log in and move money.
- A role or status change should end the target user's live session.
- A closed account should keep its username and history, and the username should not be reused.

Loans.

- A loan should move through three states: pending, approved, and rejected. Only a customer should apply.
- The amount and term should be positive. The purpose should be stored with the application.
- The administrator should set one annual interest rate and one method, simple or compound. The initial rate should be 0.00% and the initial method should be simple.
- When a customer applies, the application should copy the current policy into the loan and store the interest and total payable.
- A later policy change should affect only new loans. Existing pending and approved loans should keep the policy they were created with.
- Only a pending loan should be assigned or decided.
- Approval should credit the principal and write a loan transaction. Rejection should change only the status.

Feedback.

- Only a customer should submit feedback.
- A new feedback record should have the status new.
- A manager should mark it reviewed and write a reply.
- The customer should be able to read the reply.

Sessions and audit.

- Each session should belong to one user and one handler process.
- The server should check the session table for every authenticated request.
- Deactivation, account closure, role change, and logout should remove the session.
- Every state-changing operation should post an audit record. The logger thread should write a timestamped line to the audit file.

1.5 Data requirements

Entity	Key	Fields
User	id	username, name, role, active flag, salt, password hash, balance, created time
Loan	id	customer id, employee id, principal, term, status, interest method, rate, interest amount, total payable, purpose, applied and decided times
Transaction	id	user id, type, amount, balance after, timestamp, note
Feedback	id	user id, status, timestamp, message, reply
Session	slot	user id, handler pid, active flag
Audit entry	position	timestamp, user id, action text

A customer can have loans, transactions, and feedback. Employees process loans. Managers assign loans, change customer status, and review feedback. A user can have at most one session. An audit entry should name the user who performed the action.

1.6 Personas

We need to support four roles: customer, employee, manager, and administrator. Figure 1 shows their main actions and the interactions between them. The client menus follow these roles.

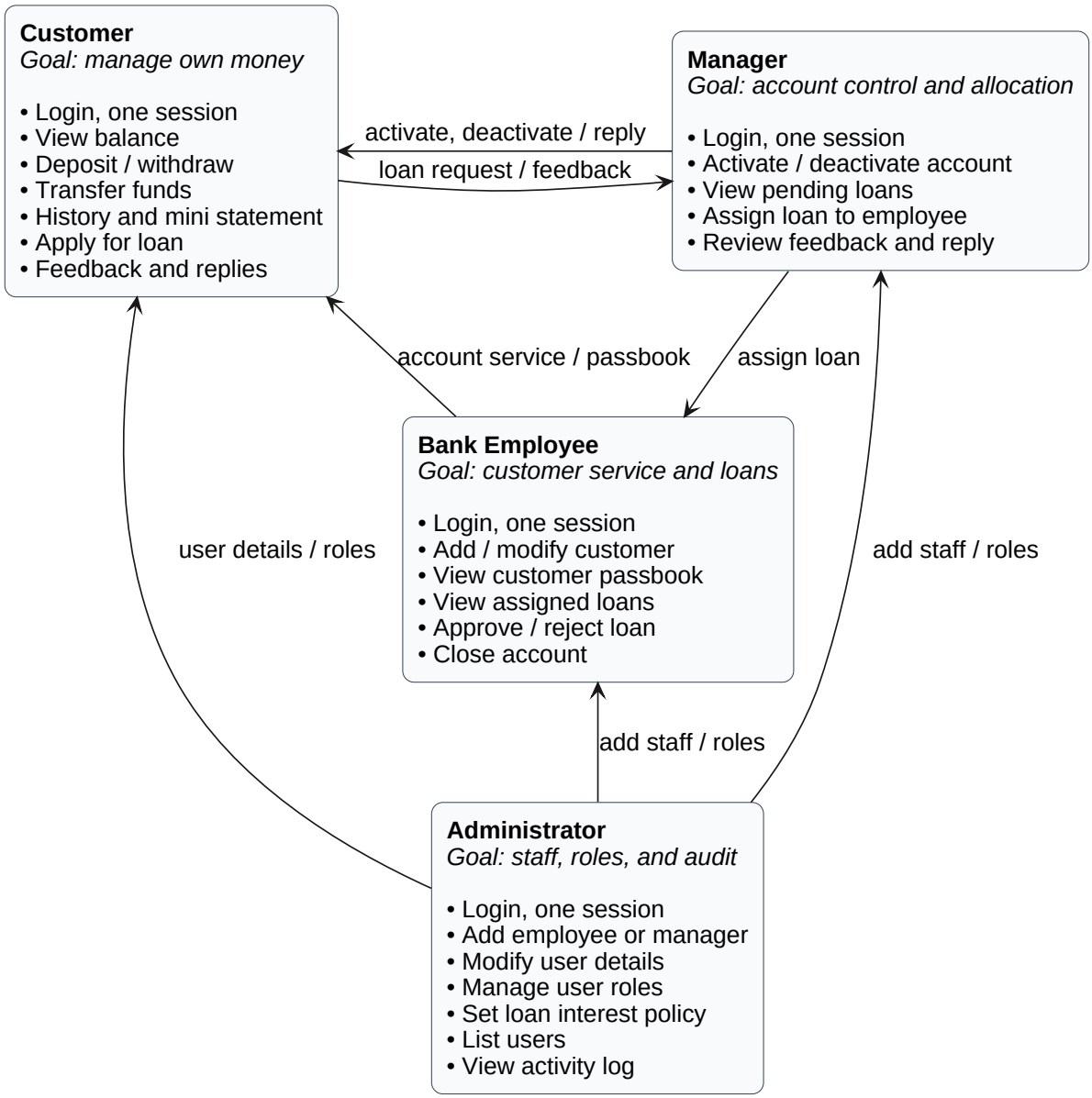


Figure 1: Role personas, their main actions, and the main interactions.

1.7 Non-functional requirements

- Concurrency: several clients should be able to work at the same time. Readers should take shared record locks, and writers should take one whole-file lock.
- Atomicity: a multi-record operation should write the complete new database or leave the previous database untouched.
- Consistency: business checks should run while the writer holds the database lock.
- Isolation: a reader should not read a record while a writer holds the whole-file lock, and it should not see the temporary file.

- Durability: the temporary file and the directory should be flushed before the server reports success.
- Capacity: the design should support 200 users, 200 loans, 2000 transactions, 200 feedback records, and 64 sessions.
- Security: the server should store salted password hashes, check roles and sessions, keep data files at mode 0600, and write an audit trail.
- Portability: the implementation should use C17 and POSIX/Linux system calls with plain files.
- Visibility: the server should print startup messages and write a timestamped audit file.

2 Technical design

2.1 System Design

We need two programs. The server process should use the data directory for storage, create the System V IPC objects, and listen on the socket. The client should connect to `127.0.0.1:9099` by default and show the role menus.

The server parent should accept a connection and fork one handler child for the whole client session. The parent should use `select` on the listening socket and a self-pipe for signals. The child should use `select` with a one-second timeout so that it can notice a shutdown even when the client is quiet. A logger thread in the parent should read audit events from the message queue.

Figure 2 shows the system components and design.

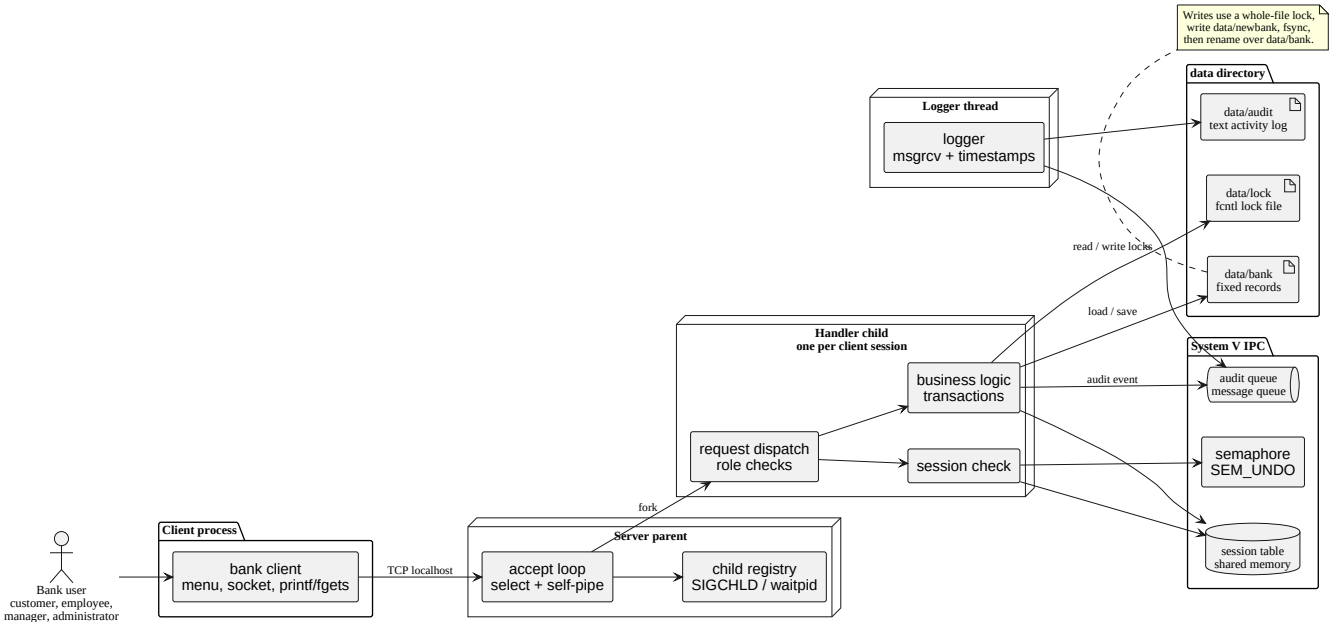


Figure 2: System components and design.

2.2 Storage design

We need to keep all persistent state in one binary file, `data/bank`. The file should contain a header and fixed-size arrays for users, loans, transactions, and feedback. The data files are extensionless.

Region	Contents
Header	File identifier, next ids, row counts, bank-wide loan rate and method.
`users[]`	Username, display name, salted hash, role, active flag, balance, creation time.
`loans[]`	Customer and employee ids, principal, term, status, rate and method snapshot, interest, total payable, purpose, timestamps.
`txns[]`	User id, type, amount, balance after, timestamp, note.
`feedback[]`	User id, status, timestamp, message, manager reply.

The header should store a file identifier, the next id for each table, and the number of used rows. A user record should store the role, active flag, username, display name, salt, password hash, balance, and timestamps. The balance is part of the user record, so one row represents a customer account. A transaction record should store the user id, type, amount, balance after the movement, time, and note. A transfer should create two rows, one for each party.

File	Purpose	Format
`data/bank`	Complete database	Binary `struct bank`
`data/newbank`	Staging file for a commit	Same binary layout
`data/lock`	Lock target	Empty file used only by `fcntl`
`data/audit`	Activity log	Timestamped text lines

The server should write the complete staging file, flush it, and rename it over `data/bank`. We should keep the lock file separate, so a rename does not affect locks.

2.3 Runtime design

- Read locks: a reader should take a shared `fcntl` lock on the byte range for the record in `data/lock`, read the record from `data/bank`, and release the lock. Reads lock the separate lock file because a writer replaces `data/bank`.
- Write locks and commit: a writer should take an exclusive lock on the whole `data/lock` file, load the database, run the business checks, and change the copy in memory. It should write the complete image to `data/newbank`, call `fsync`, rename it over `data/bank`, call `fsync` on the directory, and release the lock. The rename leaves either the old complete file or the new complete file, so a crash cannot leave a half-written database and a journal is unnecessary. The database is small, so one whole-file lock is simpler than a record-level write transaction.
- Sessions: the server uses System V shared memory to store the session table, and one semaphore protects it. Login should add one slot per user, every request should check the slot, and logout, deactivation, account closure, role change, or child exit should remove it.
- Processes and signals: the parent should accept connections and fork one handler child per client. It should use `select` on the listening socket and the read end of a self-pipe. Handlers for `SIGCHLD`, `SIGTERM`, and `SIGINT` should write one byte, so the parent does the work after `select` returns. The parent should read the pipe, collect exited children with `waitpid(-1, NULL, WNOHANG)`, and remove their sessions. The child should ignore `SIGPIPE` and exit if its parent changes because the server is gone.
- Requests and audit: the client and server should exchange fixed-size request and response values. The dispatcher should handle login separately and should check the session, active flag, and role for every other operation. Handlers should use `begin_write` or the read helpers, and successful writes should post an audit event to the message queue, where one logger thread writes it.

2.4 Security

The server should store passwords as a per-user salt and a 64-bit FNV-1a hash. The salt comes from the current time and process id. To verify a password, the server recomputes the hash and compares the result.

The server should create data files with mode 0600 inside a directory with mode 0700. It should check the active flag, the session table, and the role on every request. The client and server should use plain localhost TCP.

2.5 Class design

Figure 3 shows the complete class and data model. The full image is also available in the `docs` directory.

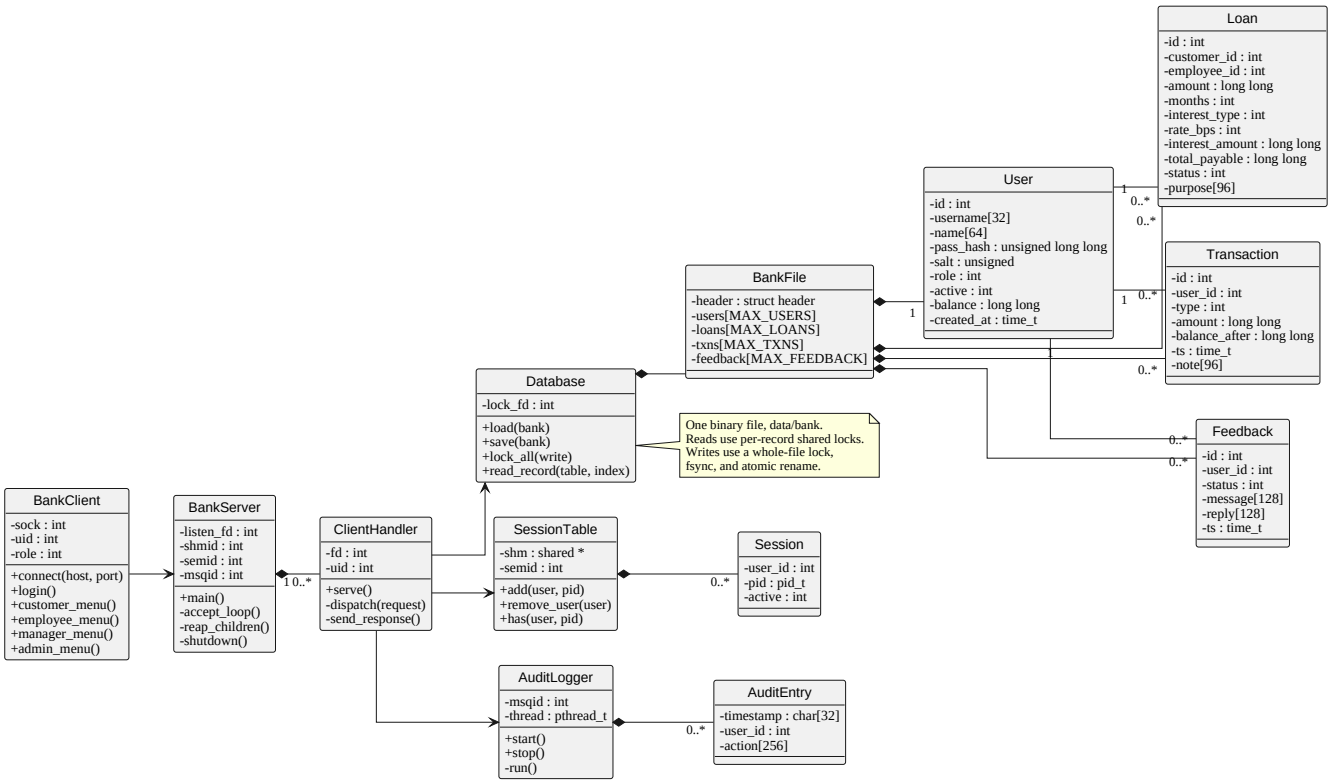


Figure 3: Complete class structure. Roles are stored in the user record and checked by the handler dispatch.

`BankClient` manages the socket and menus. `BankServer` manages the listening socket and IPC objects. `ClientHandler` runs one connection and dispatches requests. `Database` provides the data file and lock helpers, `SessionTable` provides the shared memory and semaphore, and `AuditLogger` provides the message queue and logger thread. The record classes map directly to the arrays in `struct bank`.

2.6 Sequence diagrams

Figure 4 covers login and one session per user. Figure 5 follows a transfer with the whole-file write lock and atomic rename, and Figure 6 shows the three-role loan workflow.

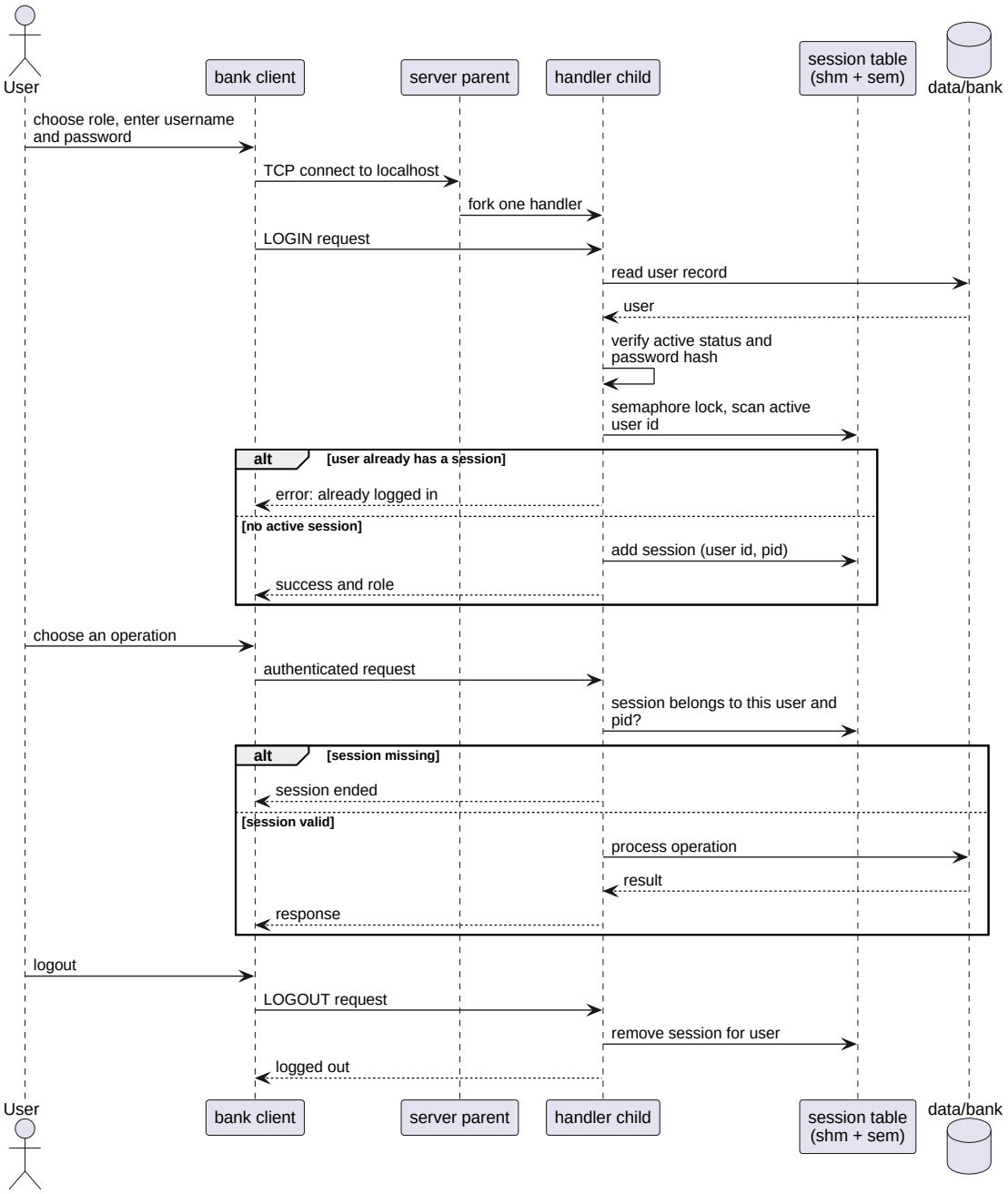


Figure 4: Login and one session per user.

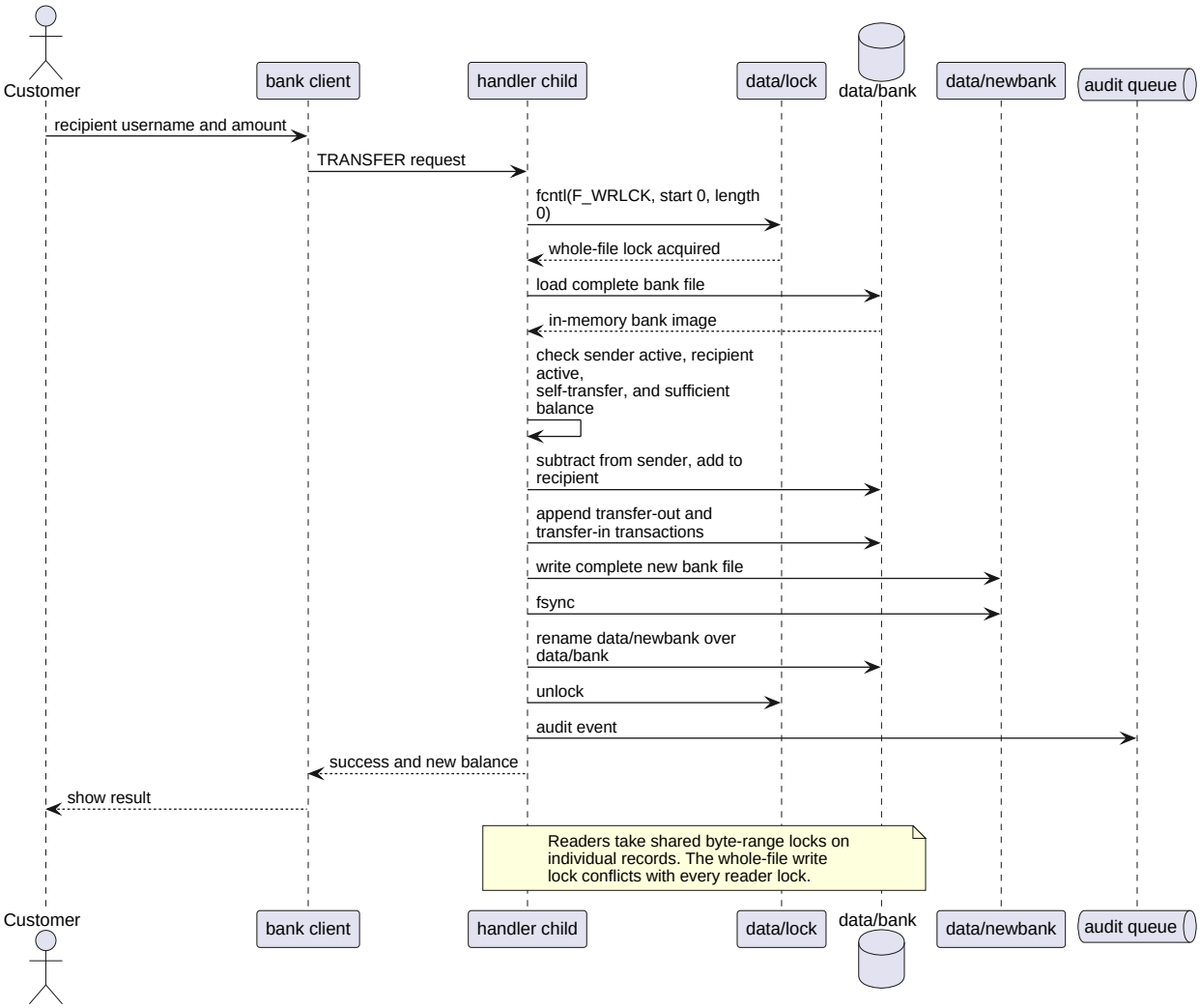


Figure 5: Atomic transfer with a whole-file write lock.

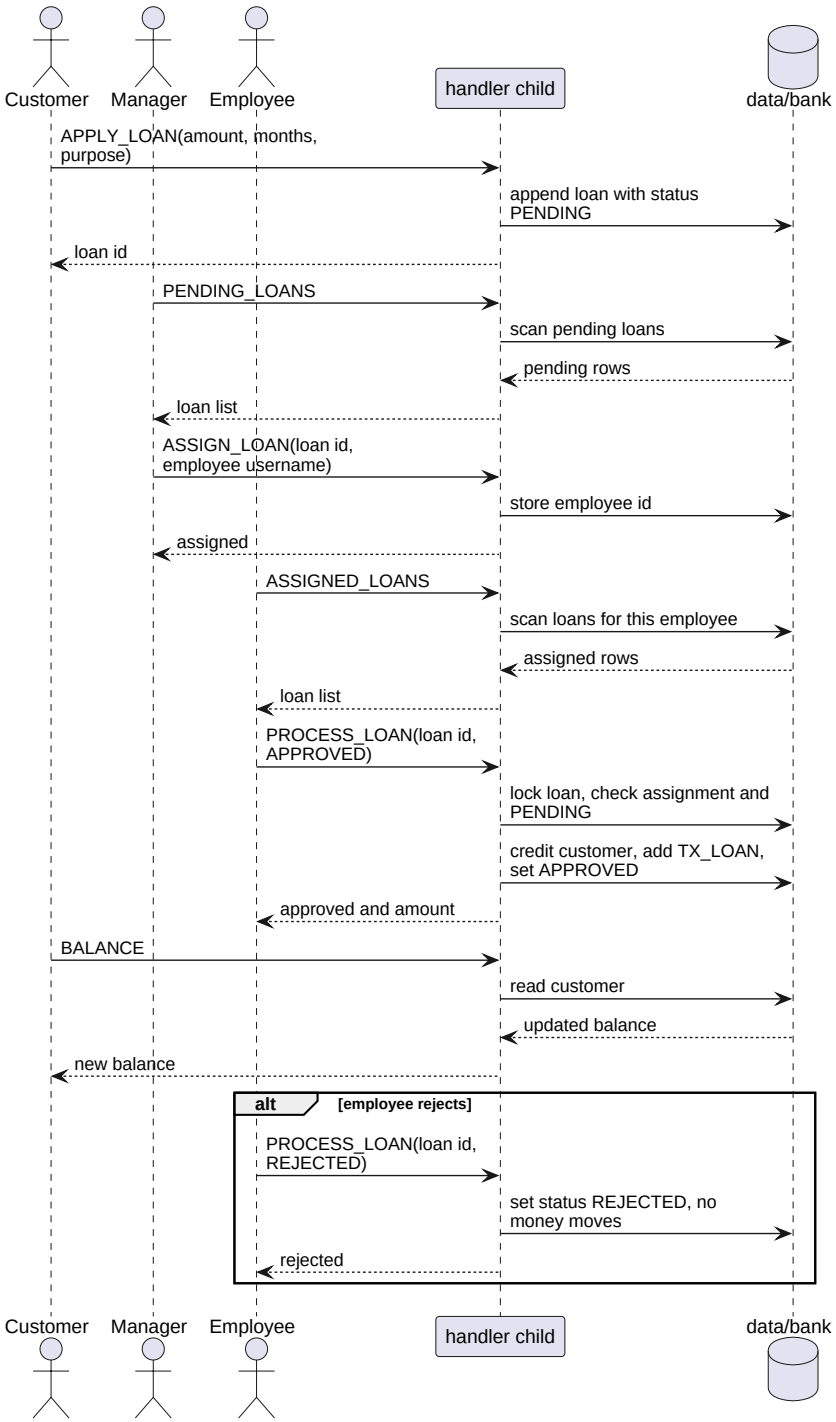


Figure 6: Loan application, assignment, and decision.

3 Implementation

3.1 Project Structure

The implementation is split into three layers. The shared layer includes storage, locking, sockets, sessions, and audit. The server layer includes the role handlers for customer, employee, manager, and administrator activity. The client layer includes the terminal menus.

Path	Responsibility
`include/`	Records, protocol, database, IPC, socket, server helper, and handler declarations.
`src/common/`	`db.c` file storage and locking, `ipc.c` sessions and audit, `util.c` shared money and write helpers, `net.c` send and receive loops.

Path	Responsibility
src/server/	server.c process and socket loop, dispatch.c request routing, and the domain modules auth.c, accounts.c, transactions.c, customers.c, loans.c, feedback.c, and admin.c.
src/client/	client.c role menus, prompts, and local menu state.
Makefile	C17 build for both programs.

Every request handled by the server follows the following set of steps:

1. The client formats a struct request and calls send_all in src/common/net.c.
2. The parent accepts the connection in src/server/server.c and forks one child. The child calls handle_request in src/server/dispatch.c.
3. The dispatcher checks the session with session_has in src/common/ipc.c, then checks the active flag and the role.
4. The handler calls begin_write and end_write in src/common/util.c. These functions call db_lock_all, db_load, db_save, and db_unlock_all in src/common/db.c.
5. The client receives the response through recv_all. The handler also posts an audit event through audit_log.

3.2 Customer functions

The customer menu is implemented in src/client/client.c. Menu item 0 clears the screen. The handlers are in src/server/auth.c, src/server/accounts.c, src/server/transactions.c, src/server/loans.c, and src/server/feedback.c.

Menu item	Handler and file	Behaviour and mechanism
Login, main 1	do_login in auth.c; session_add in ipc.c	Checks the password hash and active flag, adds one session slot under the semaphore, refuses a duplicate.
View balance, 1	do_balance in accounts.c	Reads the user record with a shared lock and prints the balance.
Deposit, 2	do_deposit in accounts.c	Takes the whole-file write lock, increases the balance, appends a DEPOSIT row, saves, and audits.
Withdraw, 3	do_withdraw in accounts.c	Rejects a non-positive amount or insufficient balance, decreases the balance, appends a WITHDRAW row, saves, and audits.
Transfer, 4	do_transfer in transactions.c	Checks the recipient, self-transfer, active status, and balance; changes both balances and adds two ledger rows in one atomic save.
Apply for loan, 5	do_apply_loan in loans.c	Validates amount and term, reads the bank-wide rate and method, calculates interest, and stores the principal, rate, interest, and total payable in a PENDING loan.
Change password, 6	do_change_password in auth.c	Verifies the old hash, creates a new salt and hash, saves, and audits.
View history, 7	do_history in transactions.c	Reads the customer's transaction records with shared locks and prints one row per movement.
Mini statement, 8	do_mini in transactions.c	Computes deposit, withdrawal, and transfer totals and prints the last five rows.
Add feedback, 9	do_add_feedback in feedback.c	Appends a feedback message.
Feedback replies, 10	do_my_feedback in feedback.c	Reads the customer's feedback records and prints manager replies.

Menu item	Handler and file	Behaviour and mechanism
Logout, 11	`do_logout` in `auth.c`	Removes the session and returns to the login menu.
Exit, 12	client exit path	Sends a logout request and closes the socket.
Clear screen, 0	`src/client/client.c`	Clears the terminal and repaints the menu.

Account operations. Deposit and withdrawal are implemented with `begin_write`. The steps are:

1. `begin_write` takes the whole-file write lock and loads the database.
2. The handler checks the amount and the current balance in memory.
3. A successful operation updates the user record and appends one transaction.
4. `end_write` writes the complete new file to `data/newbank`, flushes it, renames it over `data/bank`, flushes the directory, and releases the lock.
5. If a check fails, the handler calls `end_write(b, 0)` and writes nothing.

The balance and the ledger row therefore change in the same file image.

Transfer and race conditions. One whole-file lock is used for a transfer because it changes two user records and two ledger rows. Under that lock, the handler:

- checks the sender and recipient;
- subtracts the amount from the sender and adds it to the recipient;
- appends an outgoing row for the sender and an incoming row for the recipient;
- saves once.

If any check fails, no file is written. Two simultaneous transfers cannot interleave their updates, and parallel transfers from different customers produce exact final balances.

History, mini statement, and read consistency. The history and mini statement are read-only. They use `db_read_header` and `db_read_txn`, and each record read takes a shared lock on its byte range. A writer holds the whole-file lock, which conflicts with these shared locks. A reader therefore sees the old complete file or the new complete file, and it cannot read a record while the writer replaces the file.

Loan application, password, and feedback.

- A loan application appends a pending record and leaves balances unchanged.
- A password change verifies the old hash before it writes the new salt and hash.
- Feedback is an append-only record with an optional manager reply.

All three use the same lock, validation, atomic save, and audit path.

3.3 Bank employee functions

The employee menu is implemented in `src/client/client.c`. The handlers are in the `customers`, `loans`, `transactions`, and `accounts` modules.

Menu item	Handler and file	Behaviour and mechanism
Login, main 2	`do_login` in `auth.c`	Same hash, active flag, and one-session check as every role.
Add customer, 1	`do_add_customer` in `customers.c`	Rejects a duplicate username, hashes the password, sets the opening balance, and adds an opening deposit row.
Modify customer, 2	`do_modify_customer` in `customers.c`	Changes the display name and/or resets the password; blank fields are kept.

Menu item	Handler and file	Behaviour and mechanism
Process loan, 3	`do_assigned_loans` and `do_process_loan` in `loans.c`	Lists assigned loans with the stored interest and total, checks the assignment and pending state, then approves or rejects.
View customer transactions, 4	`do_customer_tx` in `transactions.c`	Reads the target customer's ledger with shared locks.
View assigned loans, 5	`do_assigned_loans` in `loans.c`	Lists loans whose employee id matches the session user.
Close account, 6	`do_close_account` in `accounts.c`	Requires zero balance and no pending or approved loan; marks the account inactive and removes its session.
Change password, 7	`do_change_password` in `auth.c`	Verifies the old password and stores a new salt and hash.
Logout, 8	`do_logout` in `auth.c`	Removes the session.
Exit, 9	client exit path	Logs out and closes the client.
Clear screen, 0	`src/client/client.c`	Clears the terminal and repaints the menu.

Customer administration.

- Adding a customer checks the username under the write lock, so two employees cannot create the same name.
- The opening balance and its opening transaction are part of the same save.
- Modifying a customer changes only the fields that were supplied.
- A password reset uses the same salted hash as a self-service change.
- Closing an account re-checks the balance and loan rows under the write lock, so a transfer or loan decision cannot race with the closure.

Loan processing.

- The employee can list only the loans assigned to them.
- A decision checks that the loan is assigned to the logged-in employee and is still pending.
- Approval credits the principal and adds a loan transaction in the same atomic image. Rejection changes only the status.
- The rate and interest calculated at application are kept in the loan record if the bank-wide policy changes.
- Either decision is audited. A second employee cannot decide the same loan, and a repeated decision fails because the loan is no longer pending.

3.4 Manager functions

The manager handlers are implemented in `accounts.c`, `loans.c`, and `feedback.c`.

Menu item	Handler and file	Behaviour and mechanism
Login, main 3	`do_login` in `auth.c`	Standard login and session check.
Activate or deactivate, 1	`do_set_active` in `accounts.c`	Changes the active flag under the write lock, saves, and removes the target session when deactivating.
View pending loans, 2	`do_pending_loans` in `loans.c`	Reads pending loan records with shared locks.
Assign loan, 3	`do_assign_loan` in `loans.c`	Checks pending state and an active employee, then stores the employee id.
Review feedback, 4	`do_list_feedback` and `do_review_feedback` in `feedback.c`	Lists feedback, marks one reviewed, and stores the reply.

Menu item	Handler and file	Behaviour and mechanism
Change password, 5	`do_change_password` in `auth.c`	Verifies the old password and stores a new one.
Logout, 6 and exit, 7	`do_logout` in `auth.c`	Removes the session; exit also closes the client.
Clear screen, 0	`src/client/client.c`	Clears the terminal and repaints the menu.

Account control and session revocation.

- Deactivation calls `session_remove_user` after it saves the user record under the write lock.
- The old client may still hold its socket, but its next request fails the session check.
- Reactivation changes the same flag back, so the customer can log in again.

Loan assignment and feedback.

- For loan assignment, it is first checked that the loan is pending and the employee is active; then the employee id is written into the record.
- The employee's assigned-loan list then shows the loan.
- Feedback review marks the record and stores a reply. The customer reads it through the feedback view.
- Both paths use the shared atomic save and audit.

3.5 Administrator functions

The administrator handlers are implemented in `admin.c`, with login and password in `auth.c`.

Menu item	Handler and file	Behaviour and mechanism
Login, main 4	`do_login` in `auth.c`	Standard login and session check.
Add employee or manager, 1	`do_add_employee` in `admin.c`	Checks uniqueness, hashes the password, sets the role, and saves.
Modify user, 2	`do_modify_user` in `admin.c`	Changes a name and/or resets a password for any user.
Manage roles, 3	`do_set_role` in `admin.c`	Changes a user among customer, employee, manager, and administrator; rejects a self-change and removes the target session.
List users, 4	`do_list_users` in `admin.c`	Reads user records and prints id, username, name, role, status, and balance.
View activity log, 5	`do_view_log` in `admin.c`	Reads the tail of `data/audit`.
Set loan interest, 6	`do_set_loan_rate` in `admin.c`	Stores one bank-wide annual rate and method in the database header. New loans copy the policy; existing loans keep their stored policy.
Change password, 7	`do_change_password` in `auth.c`	Verifies the old password and stores a new one.
Logout, 8 and exit, 9	`do_logout` in `auth.c`	Removes the session; exit also closes the client.
Clear screen, 0	`src/client/client.c`	Clears the terminal and repaints the menu.

Loan interest policy.

- The administrator sets one annual rate and one method, simple or compound.
- The initial rate is 0.00% and the initial method is simple.
- `do_set_loan_rate` stores the policy in the database header under the same write lock as other changes.
- When a customer applies, `do_apply_loan` copies the rate and method into the loan record and calculates the interest and total payable.

- A later policy change affects only new loans. This snapshot rule keeps the quoted loan terms stable.

Role changes and security.

- The administrator can change roles but cannot change their own role.
- A role change saves first and then removes the target session, so the old menu cannot continue with the previous permissions.
- The user must log in again, and the dispatcher checks the new role.

3.6 Screenshots

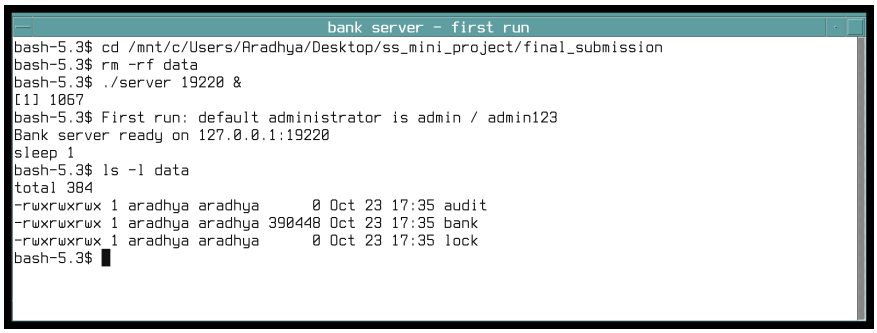


Figure 7: The first server start and the extensionless data files.

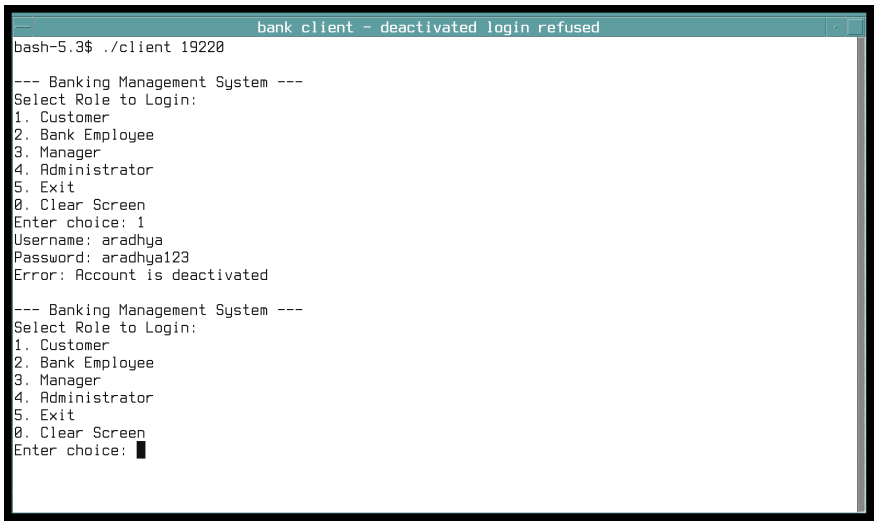


Figure 8: The server refuses a deactivated account.



Figure 9: The administrator adds employee Rahul Sharma.

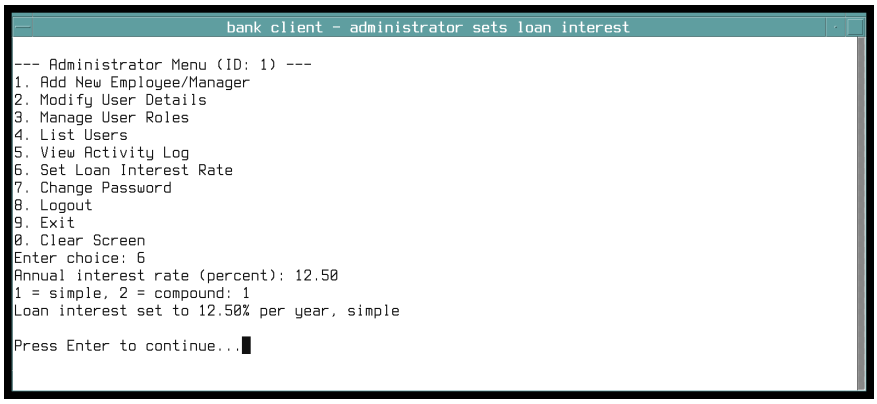


Figure 10: The administrator sets the bank-wide loan interest policy.

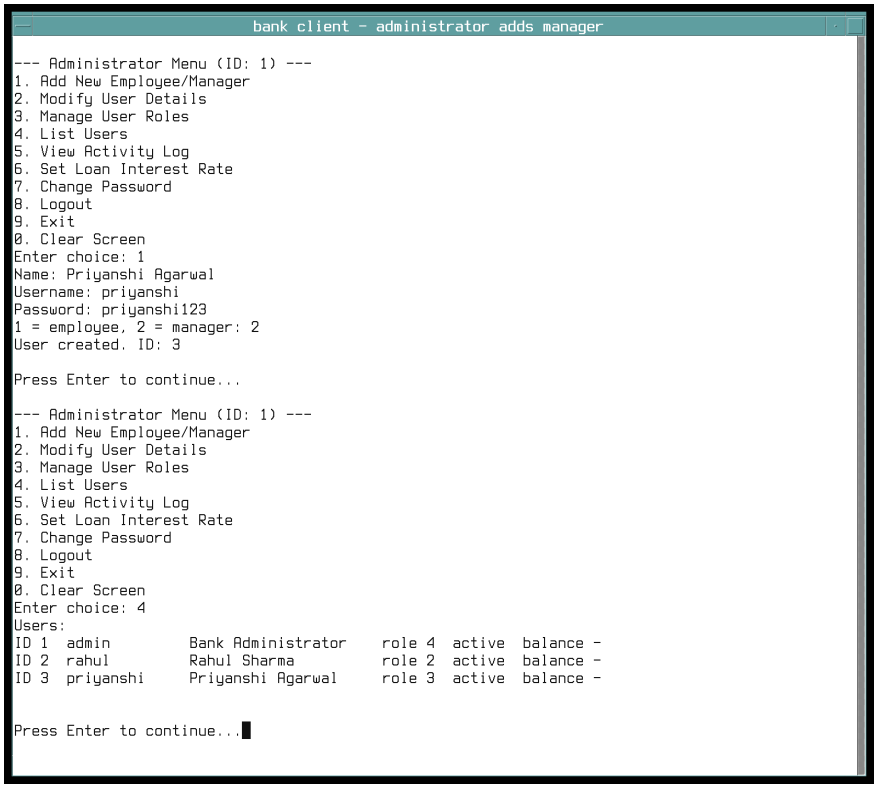


Figure 11: The administrator adds manager Priyanshi Agarwal.



Figure 12: The customer submits feedback.

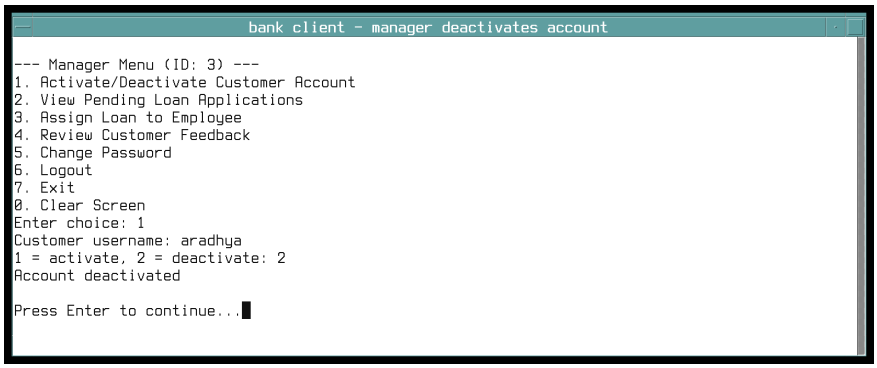


Figure 13: The manager deactivates a customer.

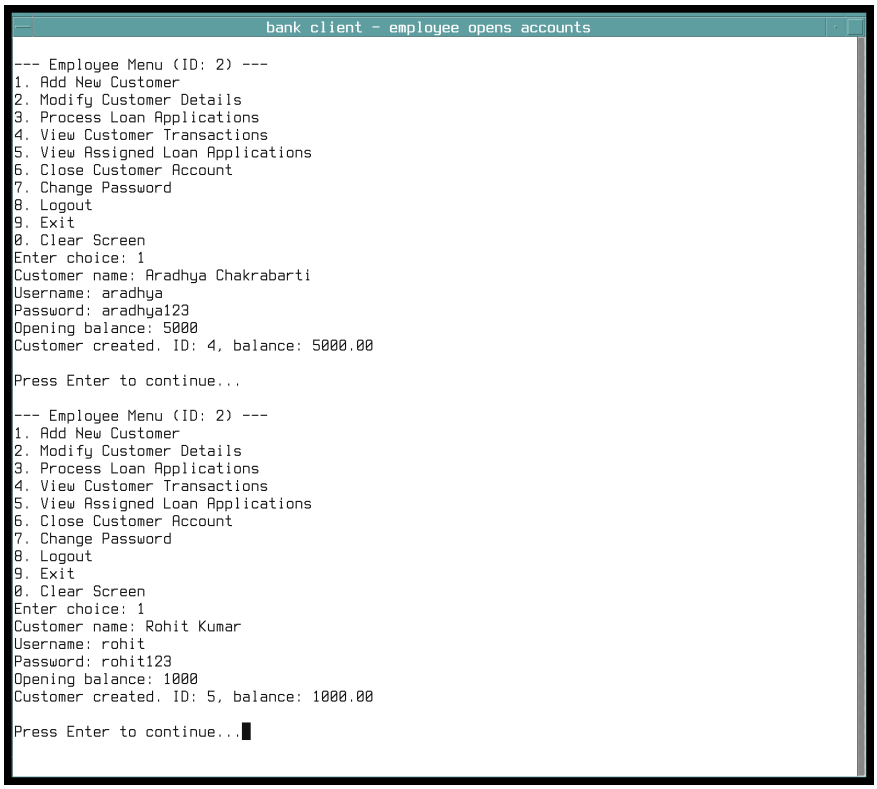


Figure 14: The employee opens customer accounts.

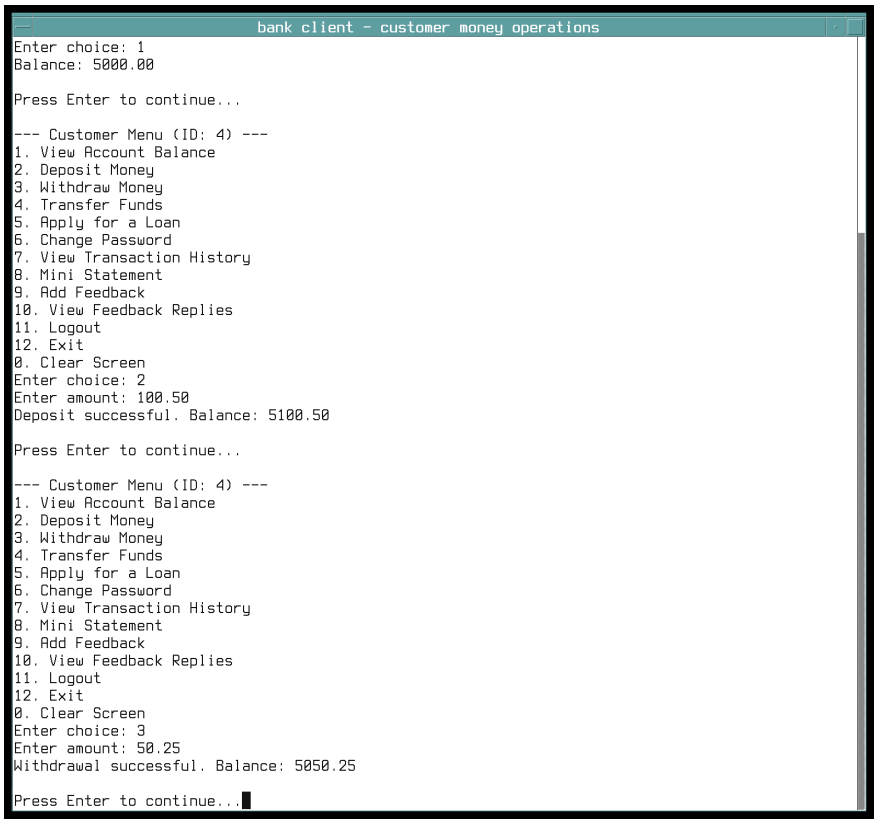


Figure 15: The customer checks the balance, deposits, and withdraws.

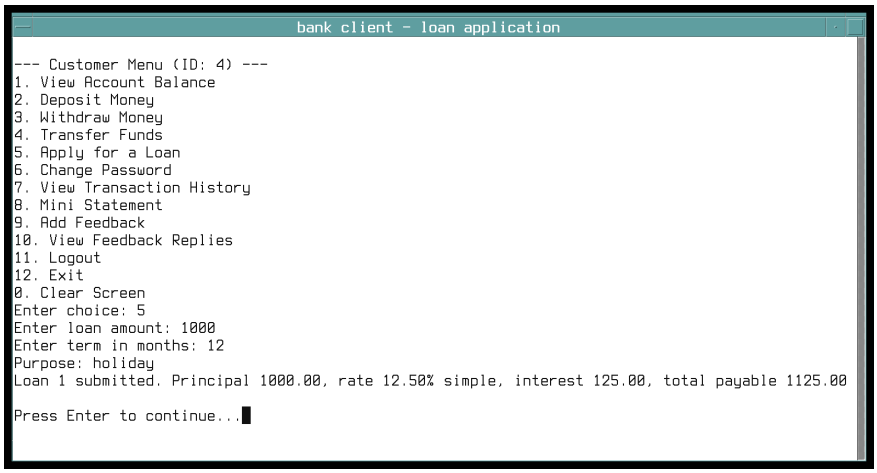


Figure 16: The customer applies for a loan with interest.

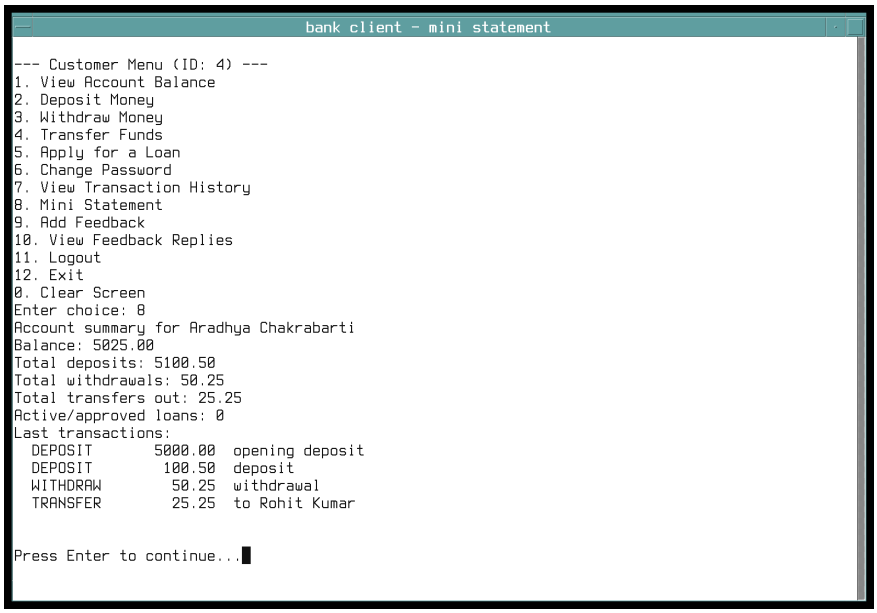


Figure 17: The customer views a mini statement.

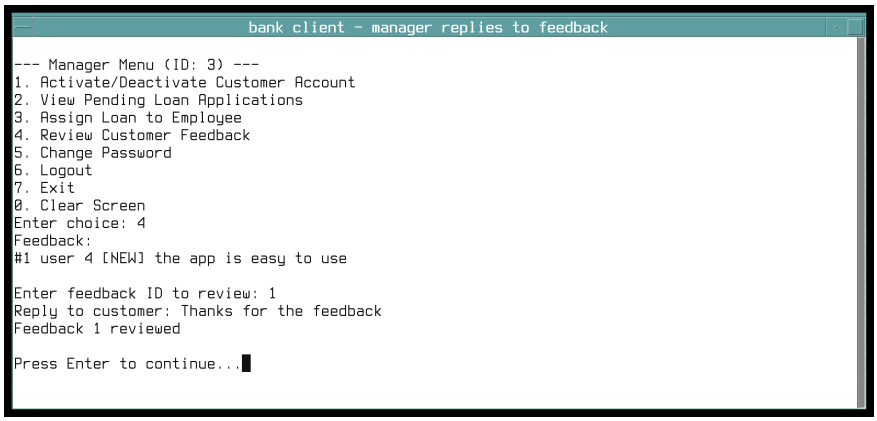


Figure 18: The manager replies to feedback.

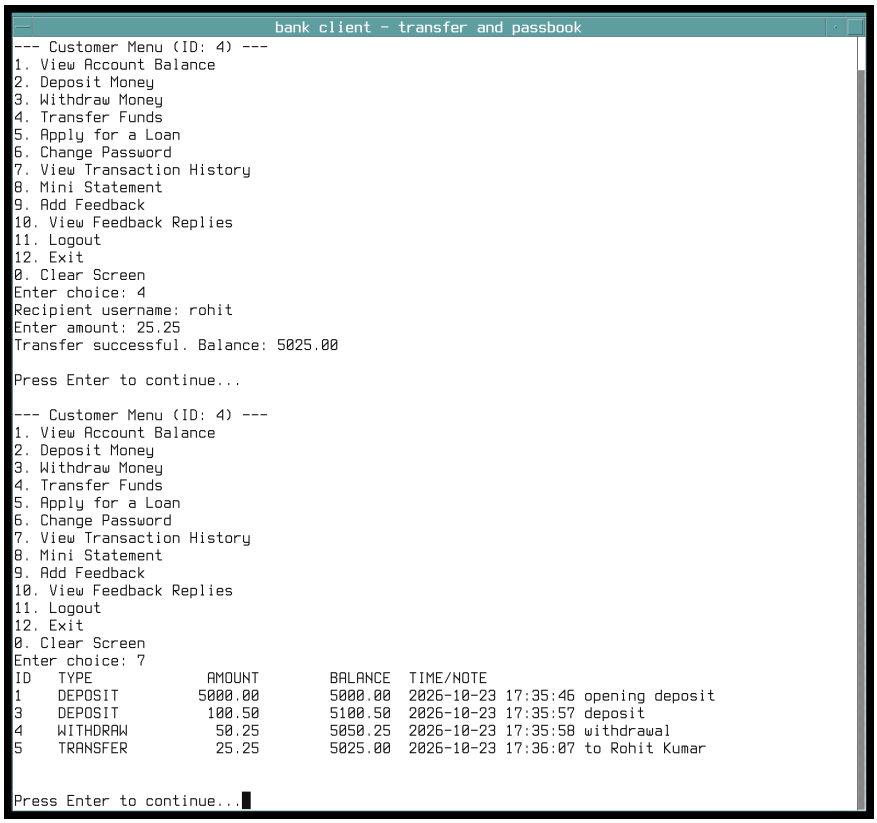


Figure 19: The customer transfers funds and views the passbook.

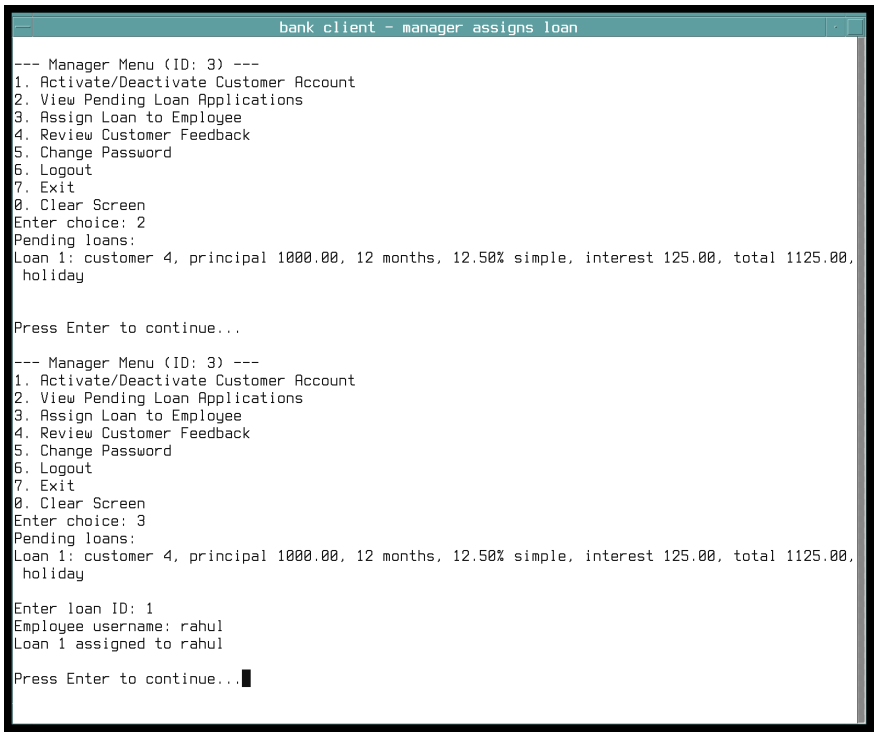


Figure 20: The manager assigns a pending loan.

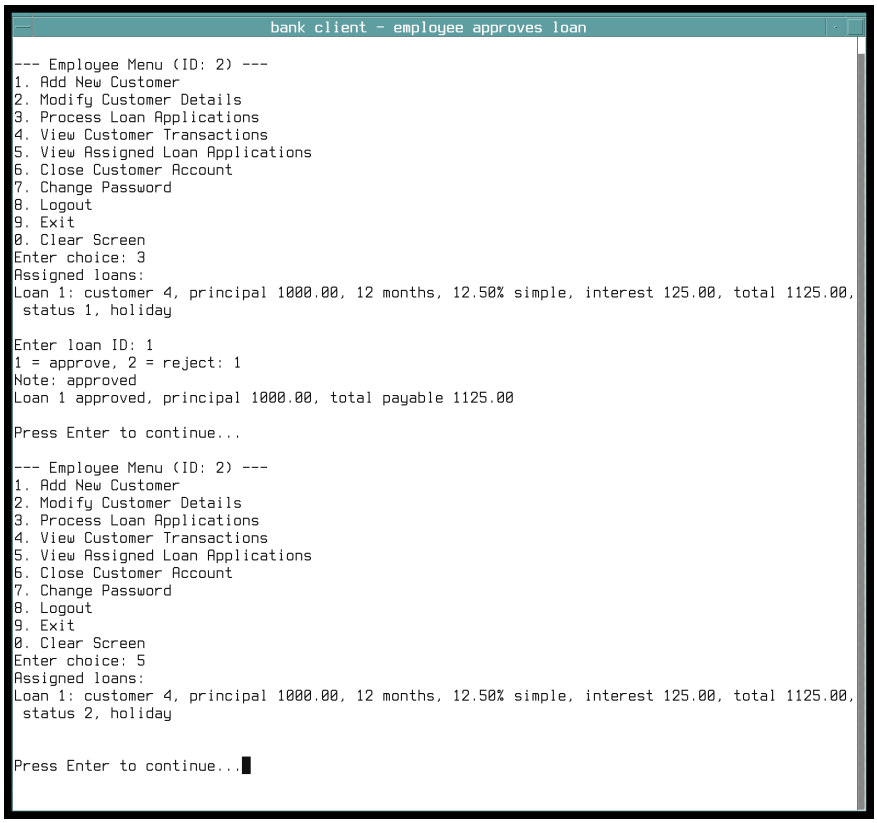


Figure 21: The employee approves the loan with the stored total payable.

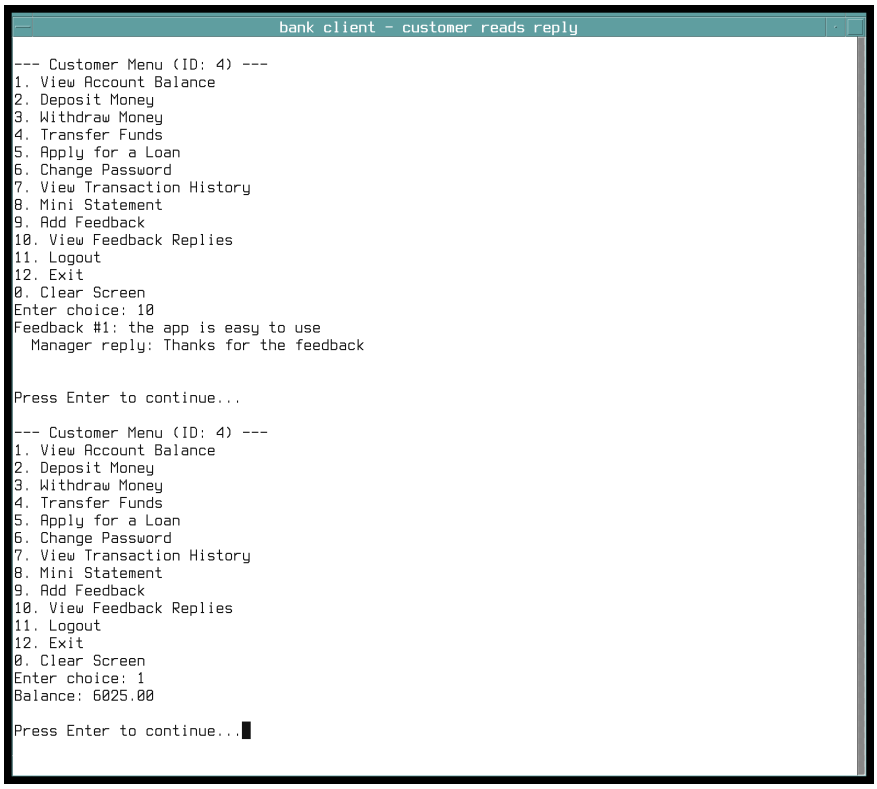


Figure 22: The customer reads the manager's reply.

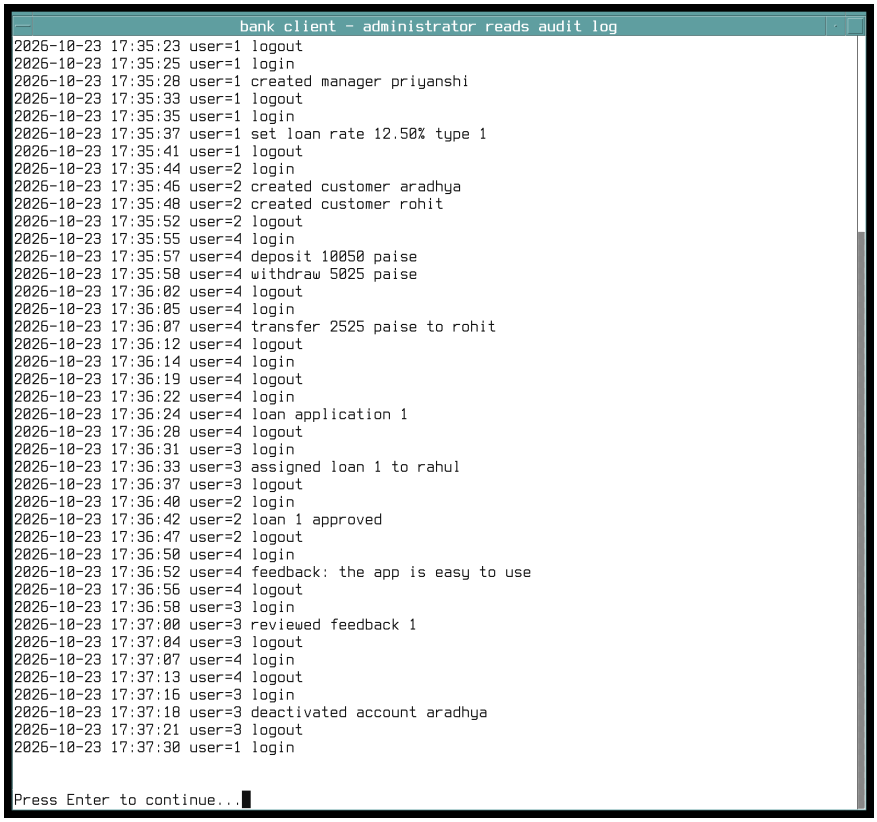


Figure 23: The administrator reads the timestamped audit log.

3.7 Extras

Three extra features were implemented.

- The mini statement (menu item 8) calls `do_mini` and shows the balance, deposit, withdrawal, and transfer totals, the pending/approved loan count, and the last five transactions.
- The feedback reply (manager item 4, customer item 10) stores a reply with `do_review_feedback` and reads it with `do_my_feedback`.
- The loan interest policy (administrator item 6, customer item 5) stores one bank-wide rate and method. `do_apply_loan` copies the policy into each new loan, and existing loans keep their original policy.